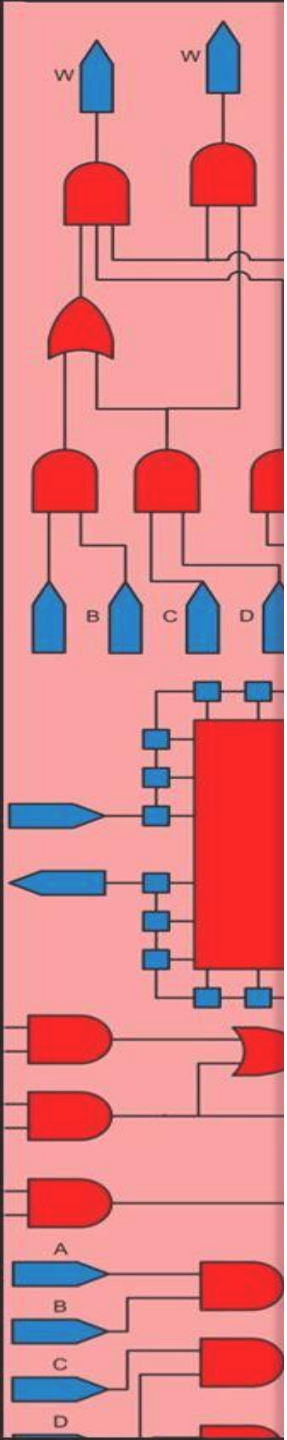


Topic 3

Sequential Components



Sequential components

3- Sequential Components

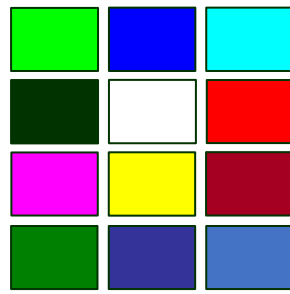
3-0 Recall

3-1 Registers

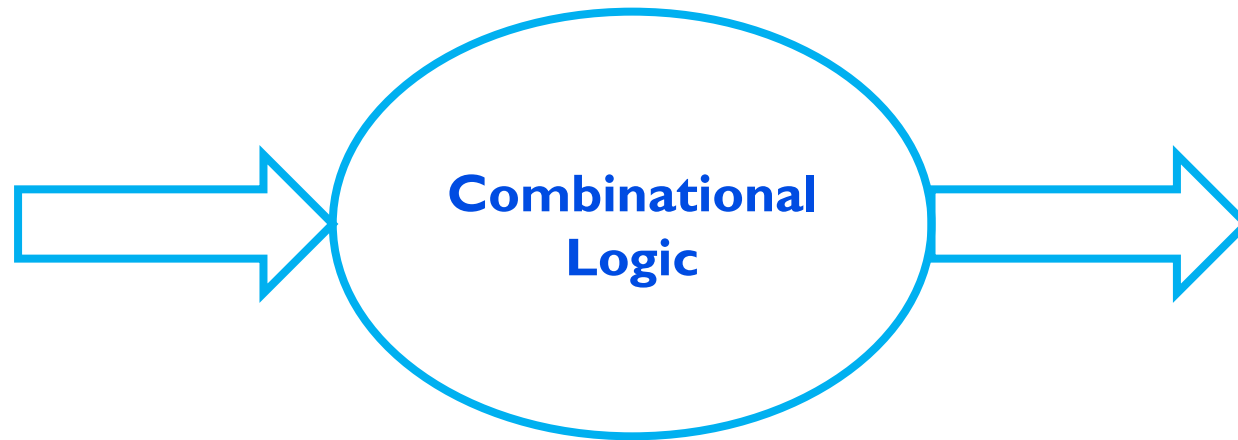
3-2 Shift Register

3-3 Counters

3-4 Register file

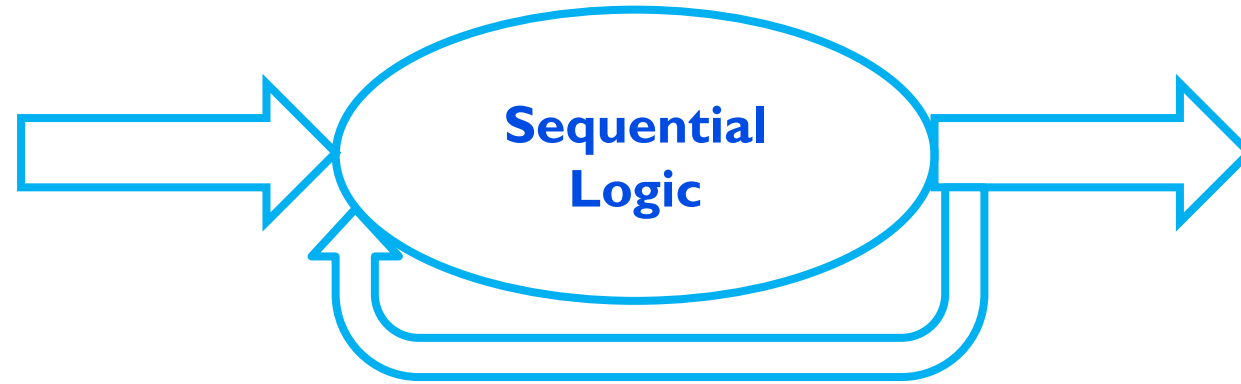
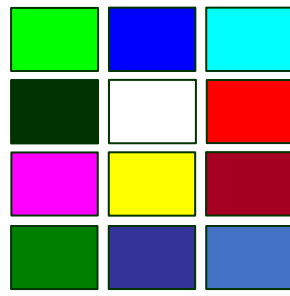


Combinational Logic

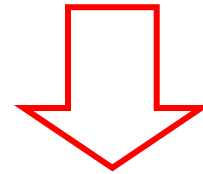


1. *Combinatorial Logic responds continuously to the values (voltage levels) at its inputs*
2. *Combinatorial logic has **no memory** to remember the past inputs or the past outputs*
3. *The past inputs or the past outputs have no impact on the present output*

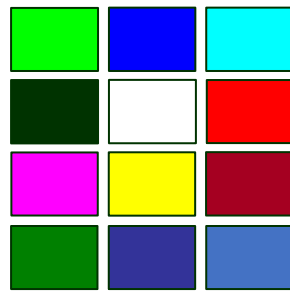
Sequential Logic / Memory



- 1. Sequential Logic also responds continuously to the values (voltage levels) at its inputs*
- 2. The past inputs or the past outputs **do have an impact** on the present output*



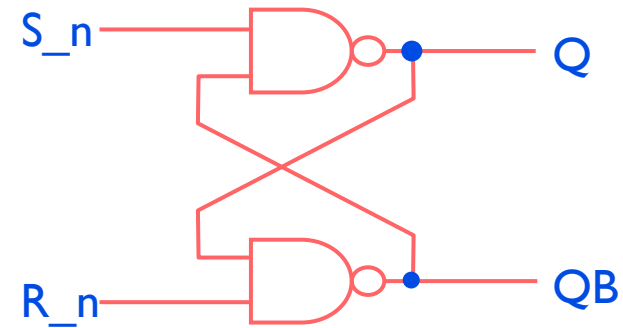
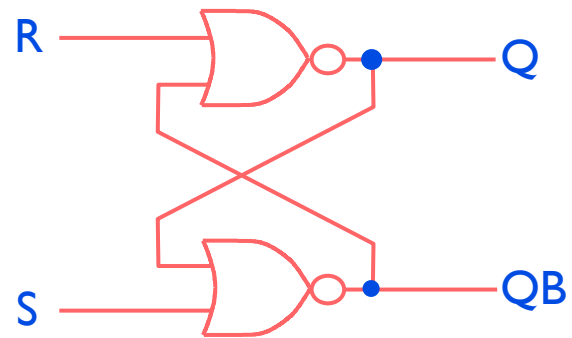
This implies that sequential logic has a memory that remembers the past inputs and outputs.



Sequential components

We know about sequential components

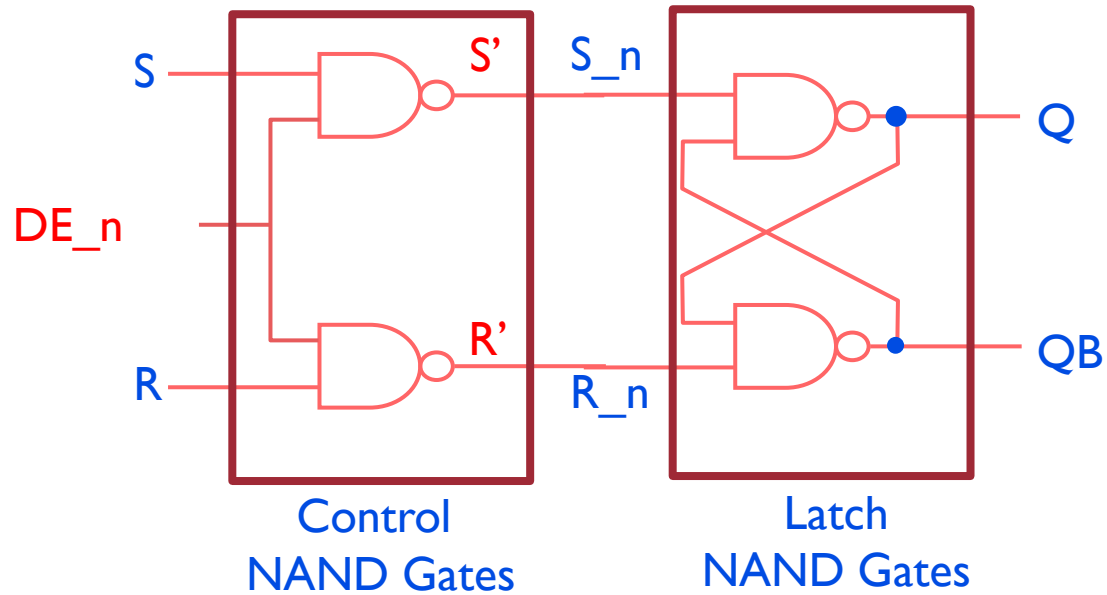
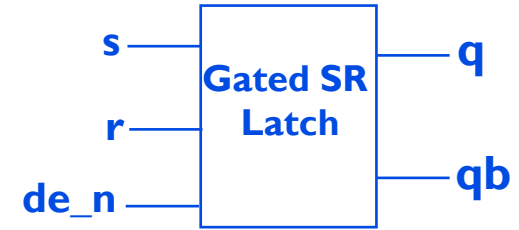
- S-R latch



Sequential components

We know about sequential components

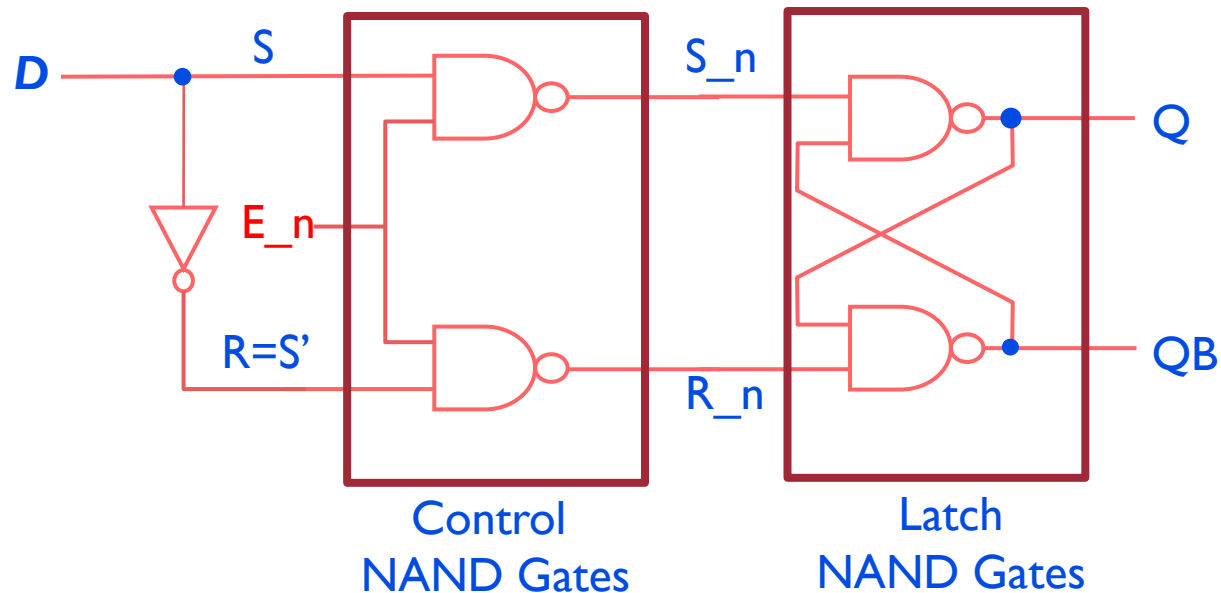
- S-R latch
- Gated S-R latch

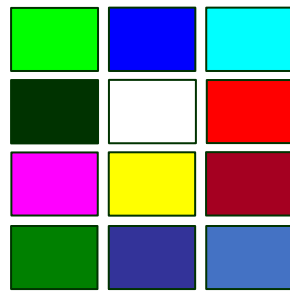


Sequential components

We know about sequential components

- S-R latch
- Gated S-R latch
- Gated D latch

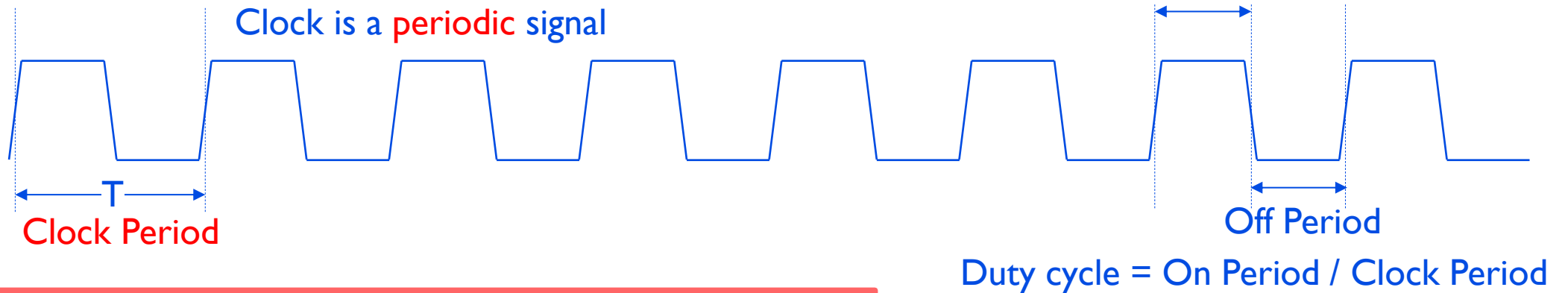




Sequential components

We know about sequential components

- S-R latch
- Gated S-R latch
- Gated D latch
- Clock concept



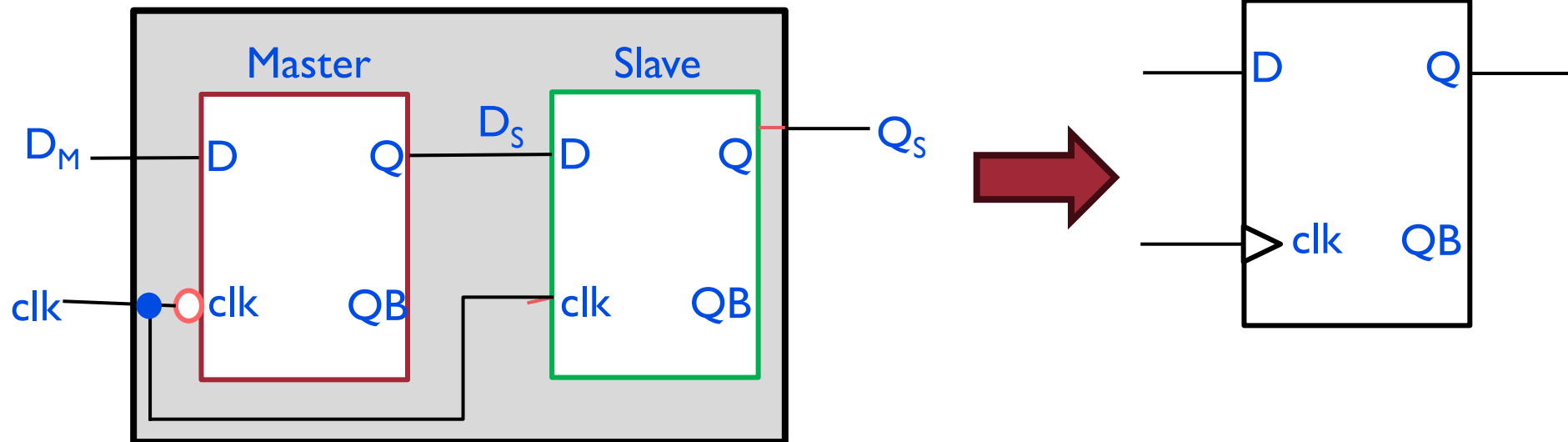
Why do we need a Clock?

- Clock periodically enables the transfer of data from input to output (synchronization)

Sequential components

We know about sequential components

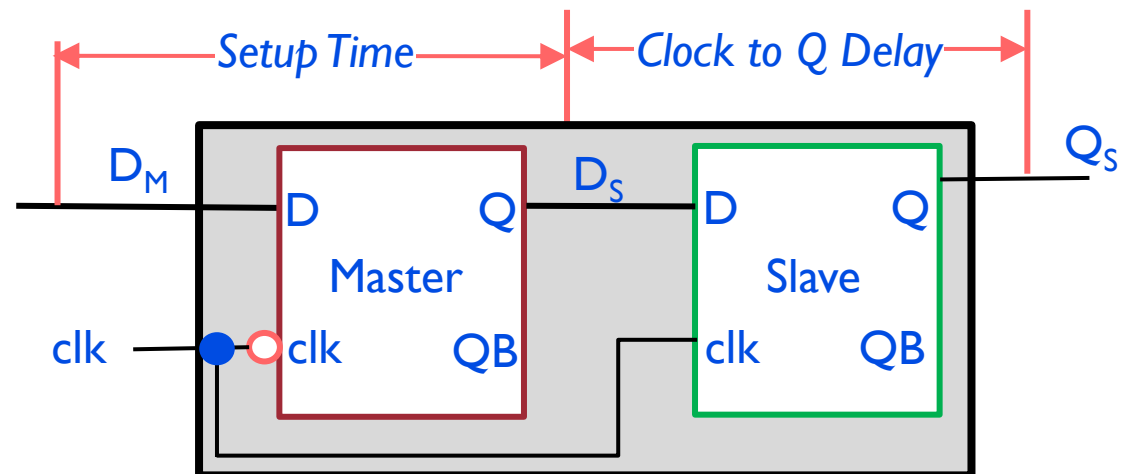
- S-R latch
- Gated S-R latch
- Gated D latch
- Clocked D latch
- D Flip-Flop



Sequential components

We know about sequential components

- S-R latch
- Gated S-R latch
- Gated D latch
- Clocked D latch
- D Flip-Flop
- Timing Constraints



Sequential components

3- Sequential Components

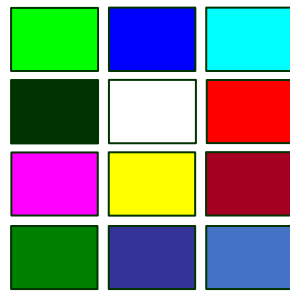
3-0 Recall

3-1 Registers

3-2 Shift Register

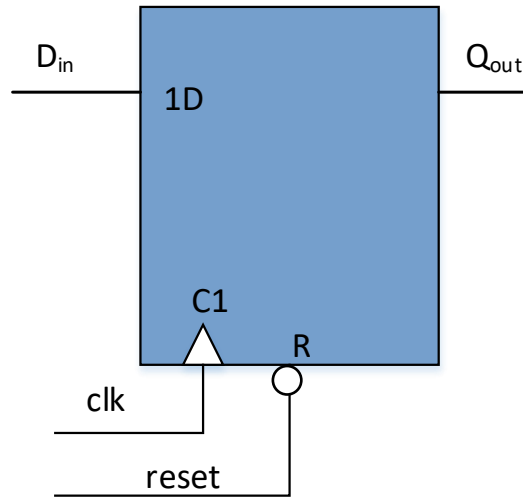
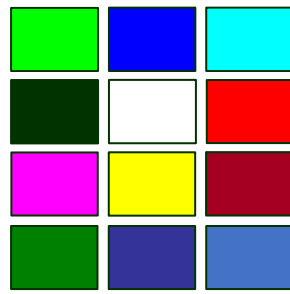
3-3 Counters

3-4 Register file

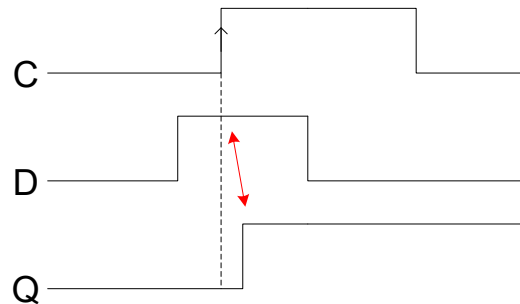


Sequential components

We have learned about sequential components.



Rising-edge D-type flip-flop with asynchronous active-low reset



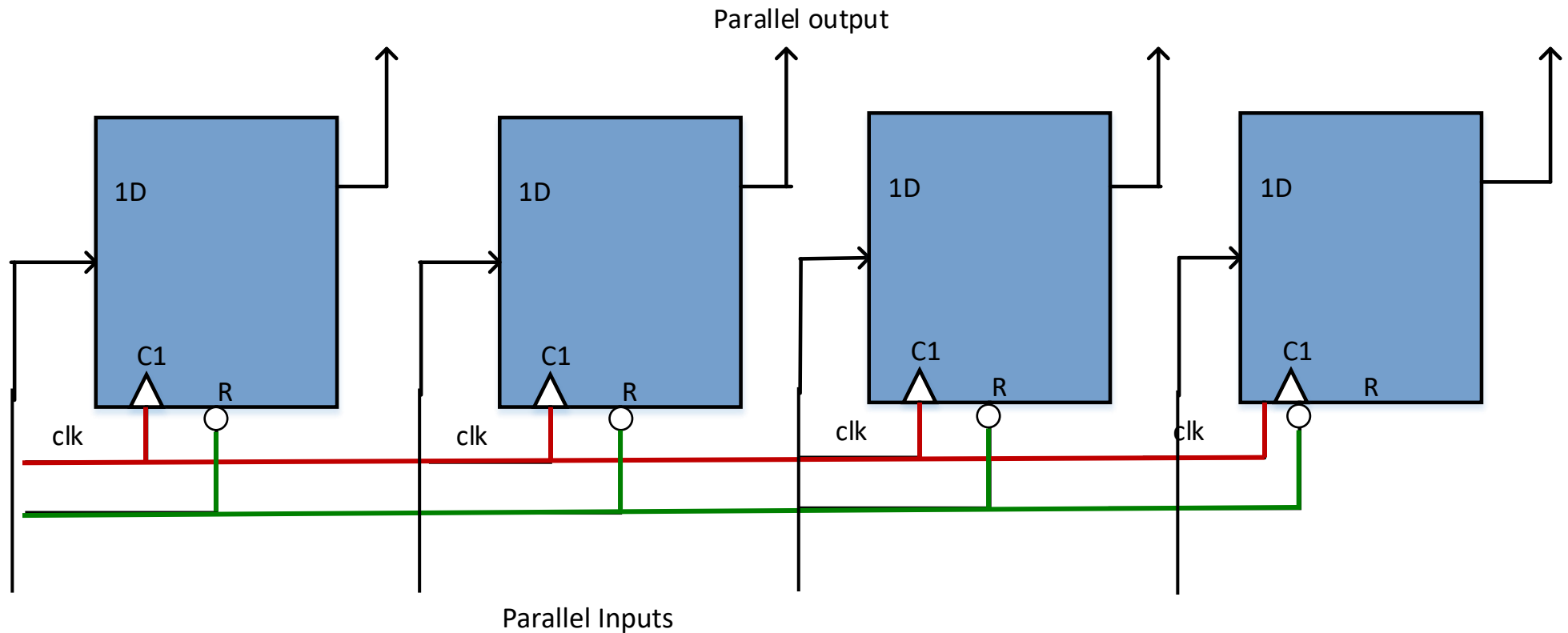
```
module DFF (  
    input logic Din,  
    input logic rst_n,  
    input logic clk,  
    output logic Qout  
);  
  
    always_ff @(posedge clk or negedge rst_n) begin  
        if (!rst_n)  
            Qout <= 1'b0;  
        else  
            Qout <= Din;  
        end  
    endmodule
```

- Use blocking assignment = in combinational logic (always_comb)
- Use non-blocking assignment <= in sequential logic (always_ff).

Sequential components

Registers

- Using a group of flip flops we can build a register.

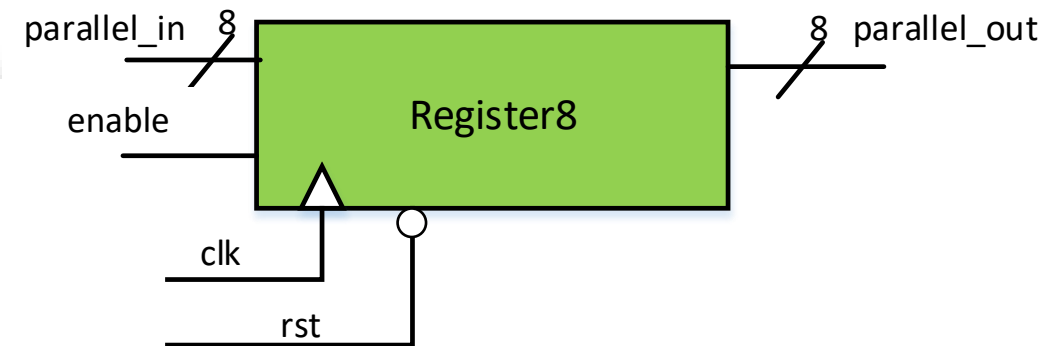


Sequential components

Registers (SystemVerilog description)

8-bit register with enable and asynchronous reset

```
module register8 (  
    input  logic [7:0] Parallel_in,  
    input  logic clk,  
    input  logic rst_n,  
    input  logic enable,  
    output logic [7:0] parallel_out  
);  
  
    always_ff @(posedge clk or negedge rst_n) begin  
        if (!rst_n)  
            parallel_out <= 8'd0;  
        else  
            parallel_out <= (enable) ? Parallel_in : parallel_out;  
        end  
endmodule
```



Sequential components

3- Sequential Components

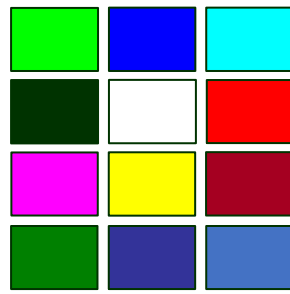
3-0 Recall

3-1 Registers

3-2 Shift Register

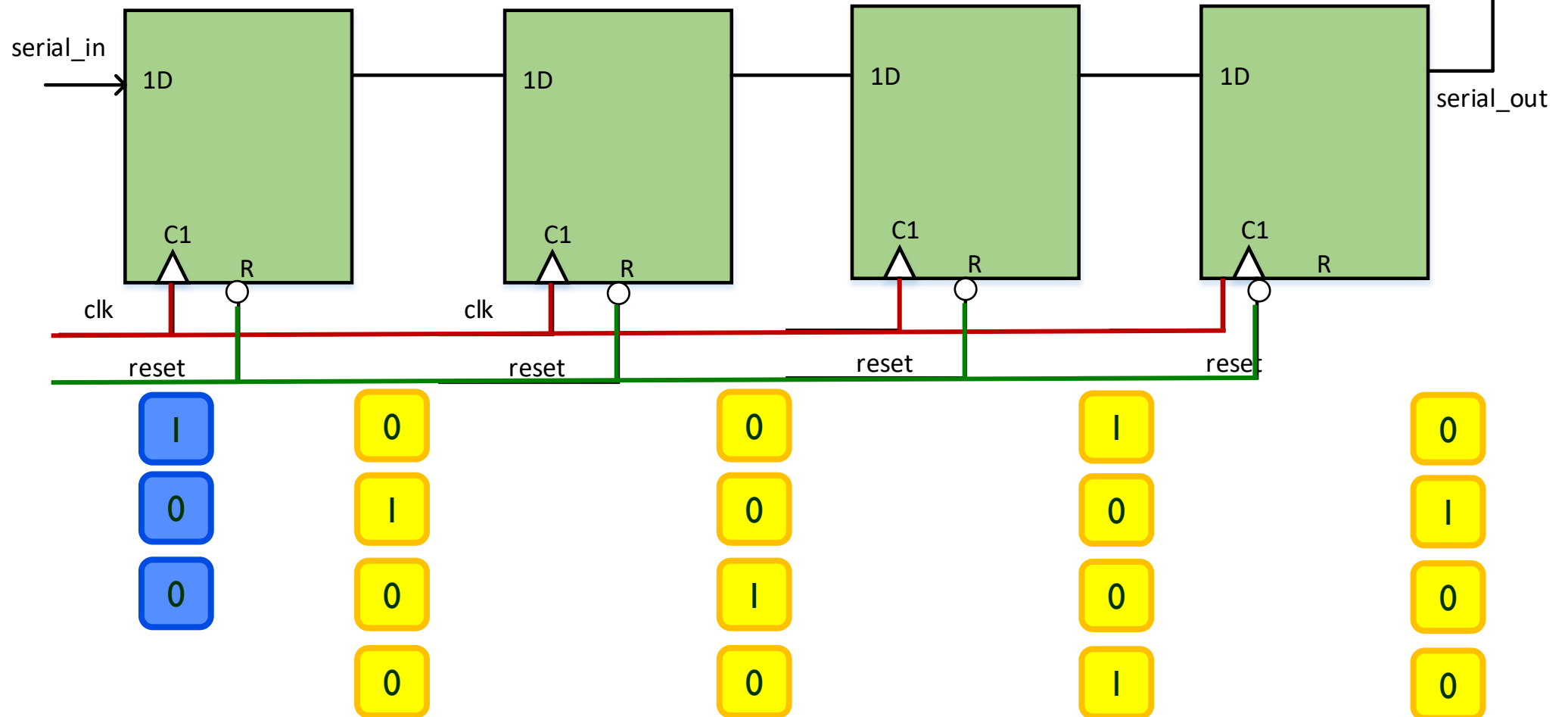
3-3 Counters

3-4 Register file



Sequential components

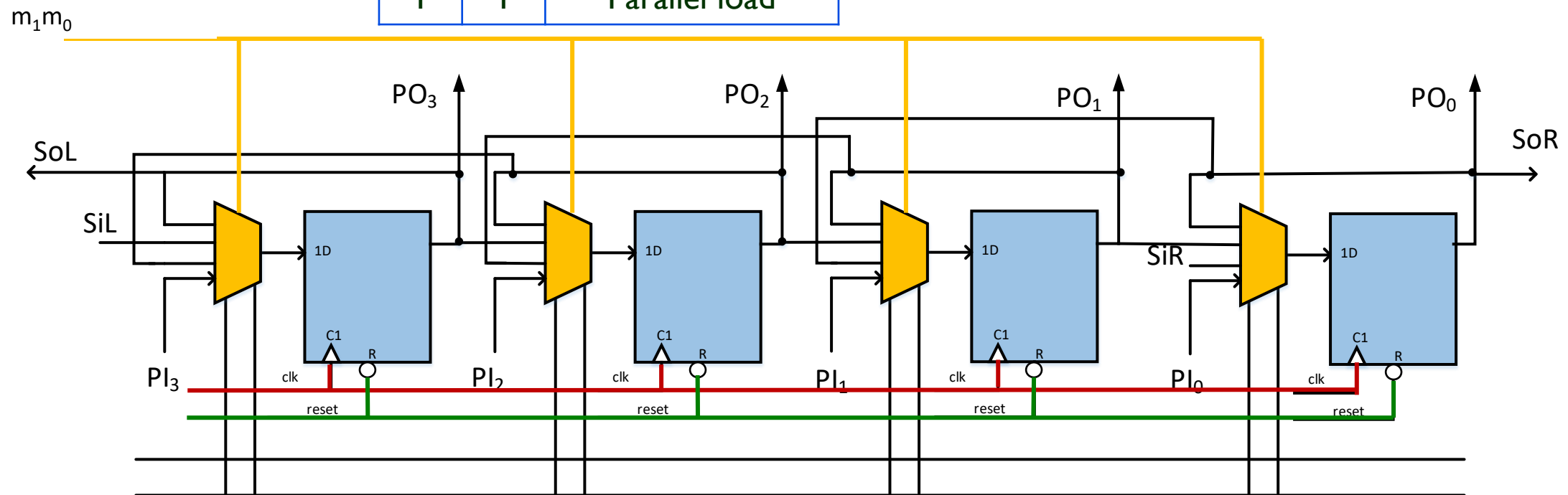
Shift Register (Functionality)



Sequential components

Universal Shift Register (Functionality)

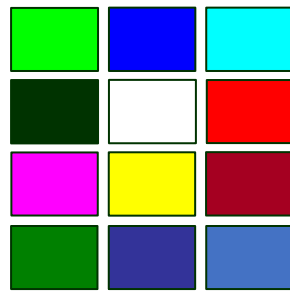
m_1	m_0	functionality
0	0	Do nothing
0	1	Right shift
1	0	Left shift
1	1	Parallel load



Sequential components

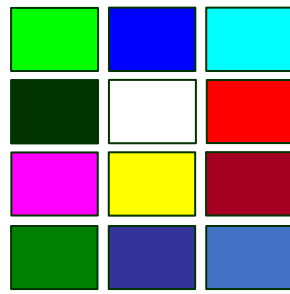
Universal Shift Register (SystemVerilog description)

```
module SR4RL (  
    input  logic [3:0] PI,  
    input  logic clk,  
    input  logic rst_n,  
    input  logic SiL,  
    input  logic SiR,  
    input  logic [1:0] m,  
    output logic SoL,  
    output logic SoR,  
    output logic [3:0] PO  
);  
  
    assign SoL = PO[3];  
    assign SoR = PO[0];  
  
    always_ff @(posedge clk or negedge rst_n) begin  
        if (!rst_n)  
            PO <= 4'd0;  
        else begin  
            case (m)  
                2'b00: PO <= PO;  
                2'b01: PO <= {SiL, PO[3:1]};  
                2'b10: PO <= {PO[2:0], SiR};  
                2'b11: PO <= PI;  
                default: PO <= PO;  
            endcase  
        end  
    end  
endmodule
```



Sequential components

Testbench for Universal Shift Register (SystemVerilog)



```
`timescale 1ns/1ns
```

```
module SR4RL_tb;
```

```
    logic [3:0] PI;  
    logic clk, rst_n, SiL, SiR;  
    logic [1:0] m;  
    logic SoL, SoR;  
    logic [3:0] PO;
```

```
    SR4RL uut (  
        .PI(PI),  
        .clk(clk),  
        .rst_n(rst_n),  
        .SiL(SiL),  
        .SiR(SiR),  
        .m(m),  
        .SoL(SoL),  
        .SoR(SoR),  
        .PO(PO)  
    );
```

```
    initial begin  
        clk = 0;  
        forever #5 clk = ~clk;  
    end
```

```
    initial begin
```

```
        rst_n = 1'b0; m = 2'b00; PI = 4'b0000; SiL = 1'b0; SiR = 1'b0;  
        #12 rst_n = 1'b1;  
        #10 m = 2'b11; PI = 4'b1010;  
        #10 m = 2'b01; SiL = 1'b1;  
        #10 m = 2'b10; SiR = 1'b1;  
        #10 m = 2'b00;  
        #17 rst_n = 1'b0; m = 2'b11; PI = 4'b1111;  
        #30 $stop;
```

```
    end
```

```
    initial begin
```

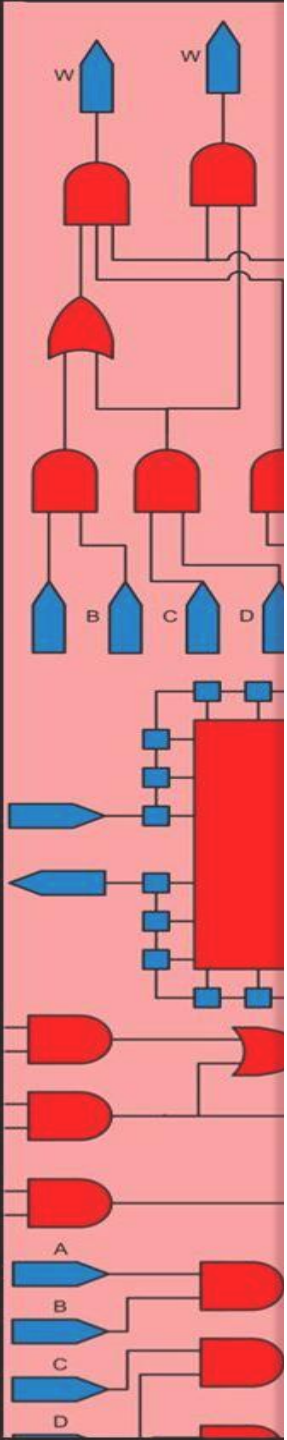
```
        $monitor("Time=%0t | clk=%b rst_n=%b m=%b PI=%b SiL=%b SiR=%b PO=%b SoL=%b SoR=%b",  
            $time, clk, rst_n, m, PI, SiL, SiR, PO, SoL, SoR);
```

```
    end
```

```
endmodule
```

always #5 clk = ~clk;

- \$display: Prints once per call
- \$monitor: Prints automatically on change



Sequential components

3- Sequential Components

3-0 Recall

3-1 Registers

3-2 Shift Register

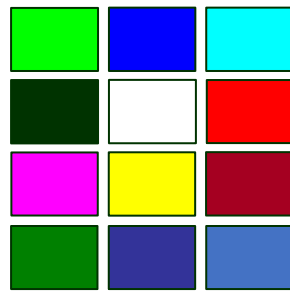
3-3 Counters

3-3-1 Up-counter

3-3-2 Frequency divider

3-3-3 Counter cascading

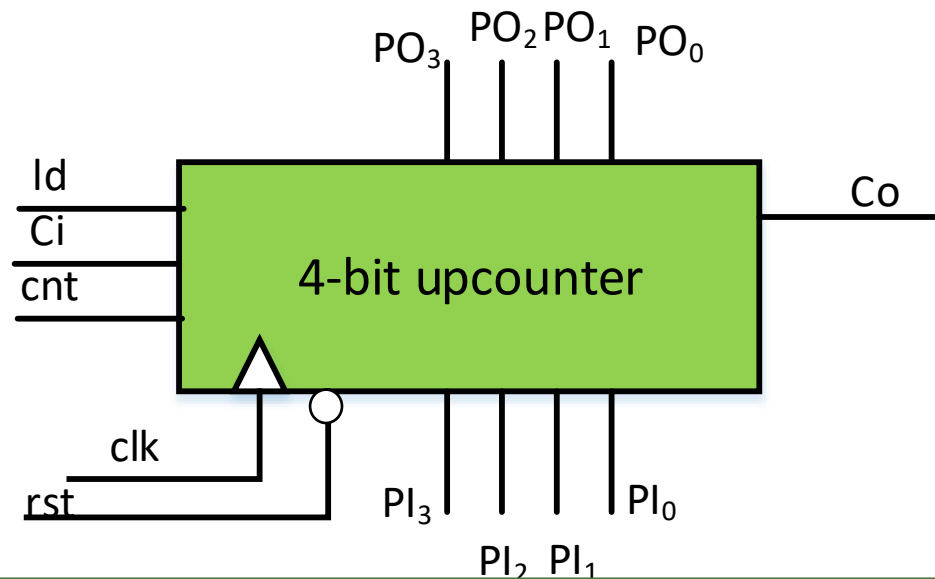
3-6 Register file



Sequential components

Counters

Up-Counter (Functionality)

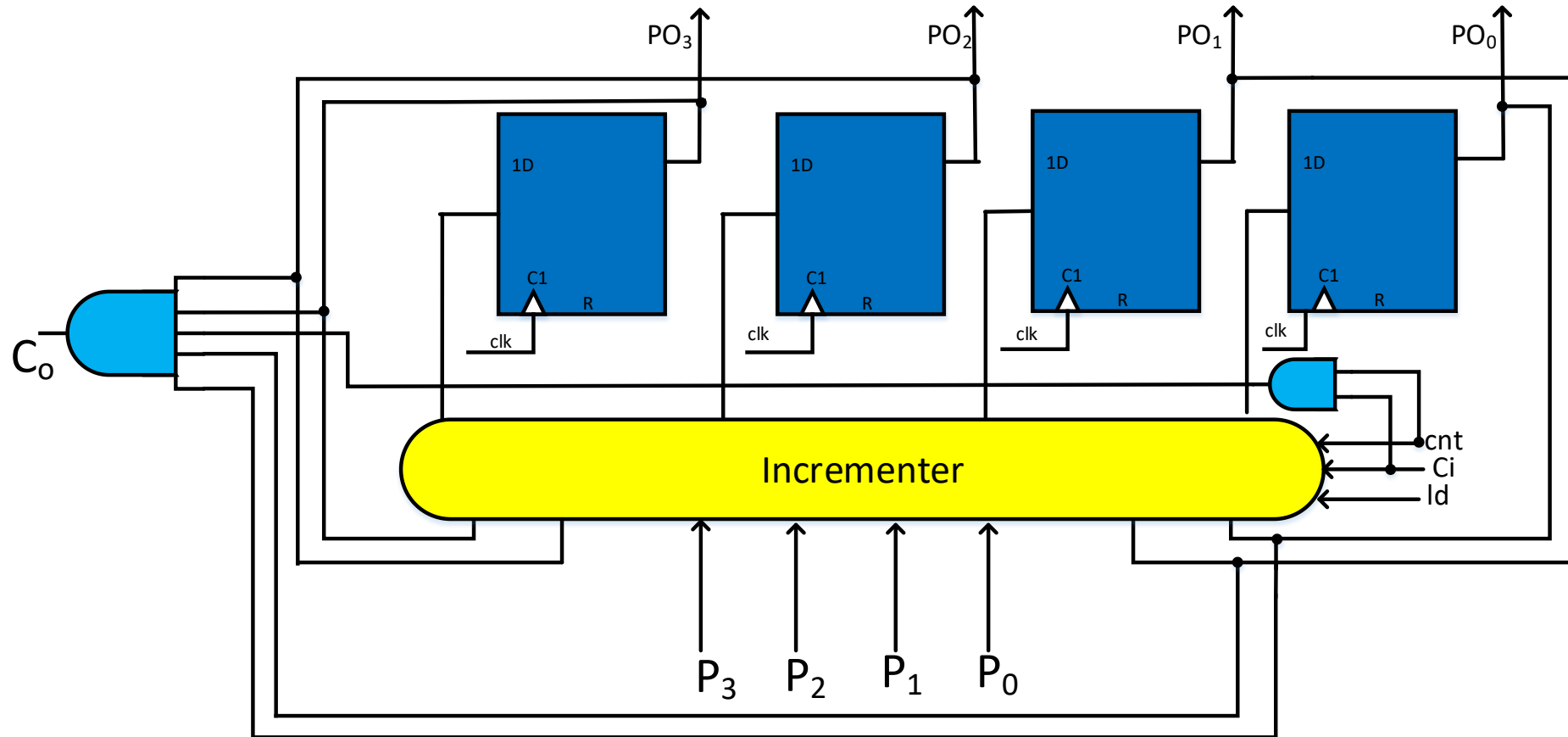


if Id is issued then the PI is passed to the PO .
If cnt and Ci are both issued then the counter starts to count up from the PI value to 15.

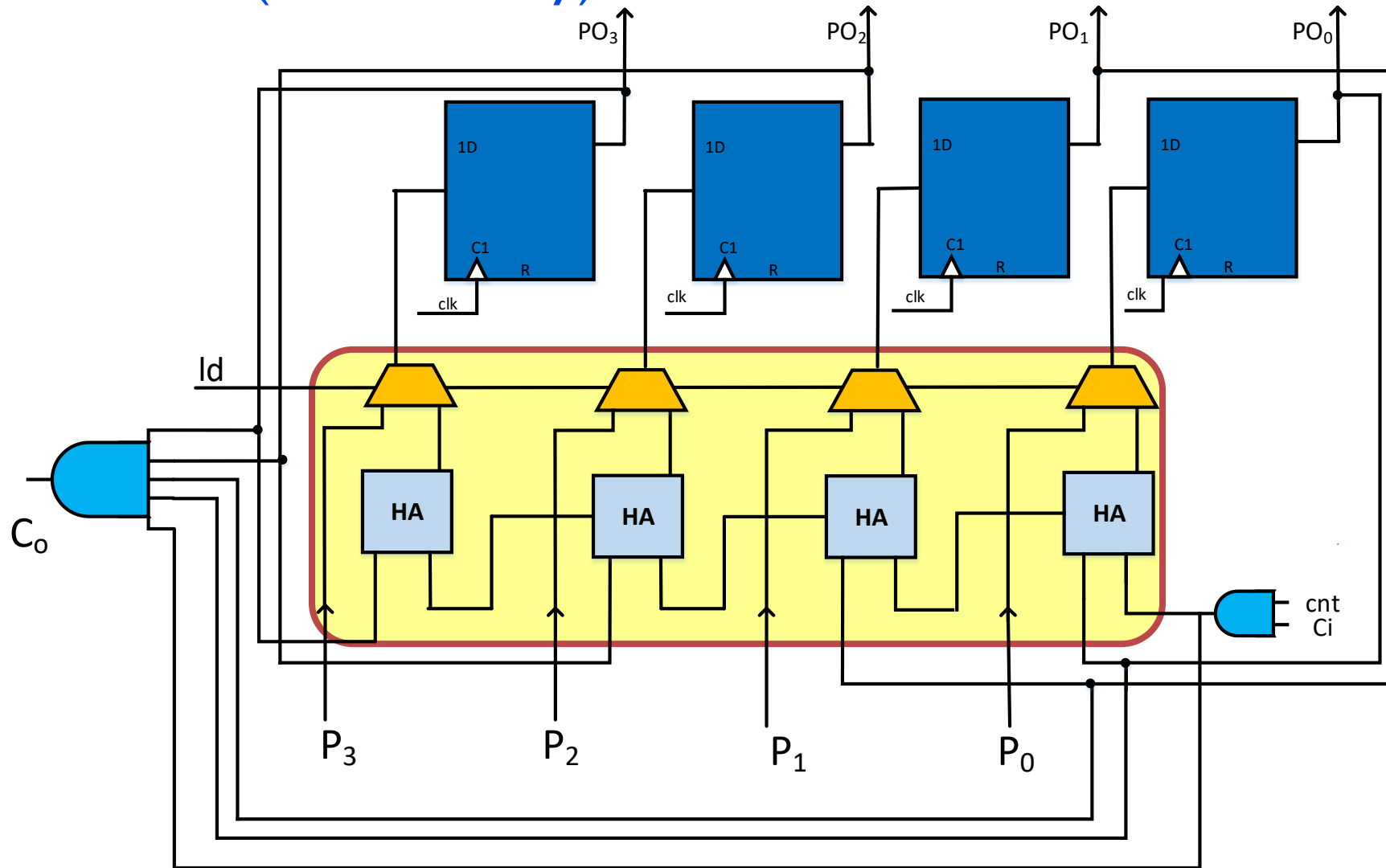
Sequential components

Counters

Up-Counter (Functionality)



Up-Counter (Functionality)

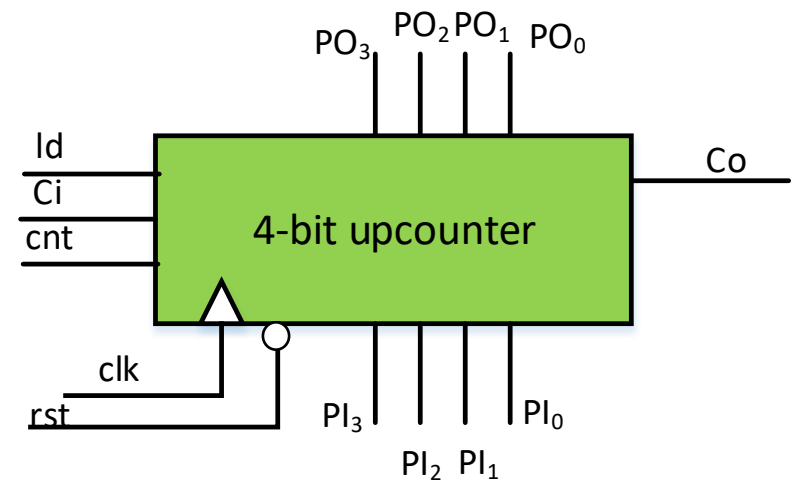


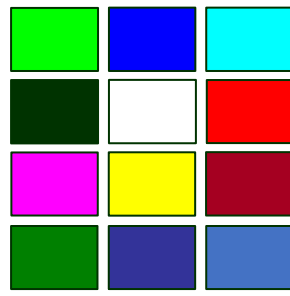
Sequential components

Counters

Up-Counter (SystemVerilog description)

```
module upcnt4 (  
    input  logic [3:0] PI,  
    input  logic clk,  
    input  logic rst_n,  
    input  logic cnt,  
    input  logic ld,  
    input  logic Ci,  
    output logic [3:0] PO,  
    output logic CO  
);  
  
    always_ff @(posedge clk or negedge rst_n) begin  
        if (!rst_n) begin  
            PO <= 4'd0;  
        end else begin  
            if (ld) begin  
                PO <= PI;  
            end else if (cnt) begin  
                PO <= Ci ? (PO + 1) : PO;  
            end  
        end  
    end  
  
    assign CO = cnt ? &{PO, Ci} : 1'b0;  
  
endmodule
```





Sequential components

3- Sequential Components

3-0 Recall

3-1 Registers

3-2 Shift Register

3-3 Counters

3-3-1 Up-counter

3-3-2 Frequency divider

3-3-3 Counter cascading

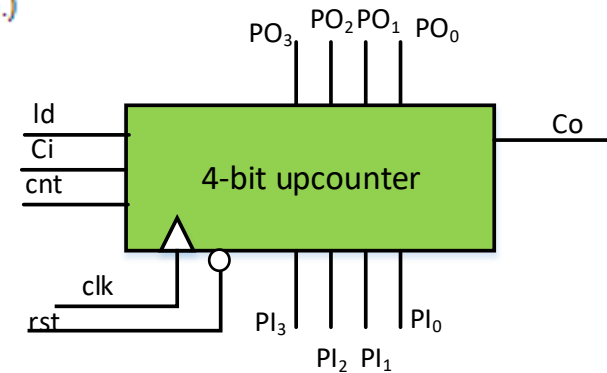
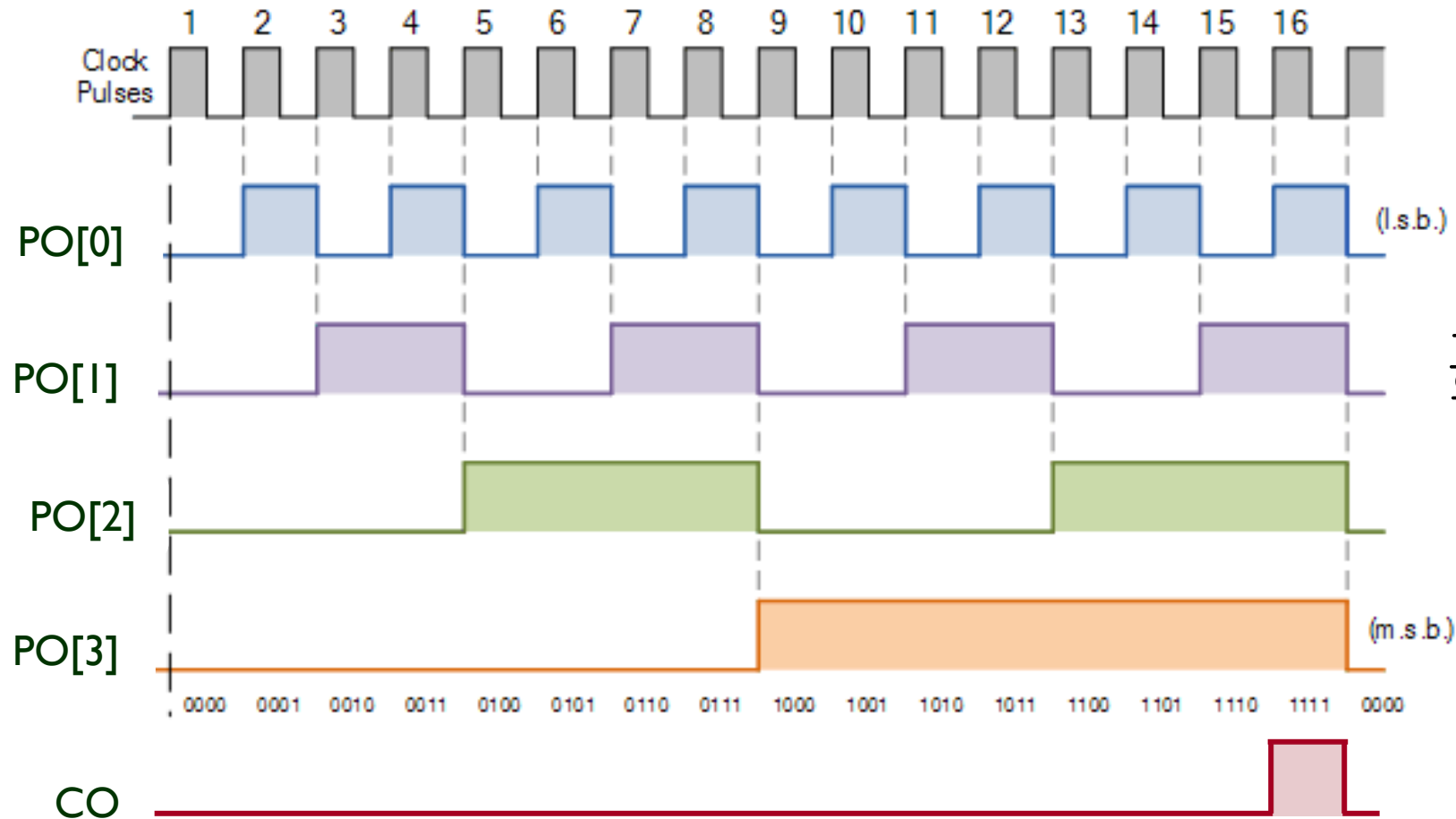
3-4 Register file

Sequential components

Frequency divider (Functionality)

The 4-bit counter can divide the clk frequency by factors of

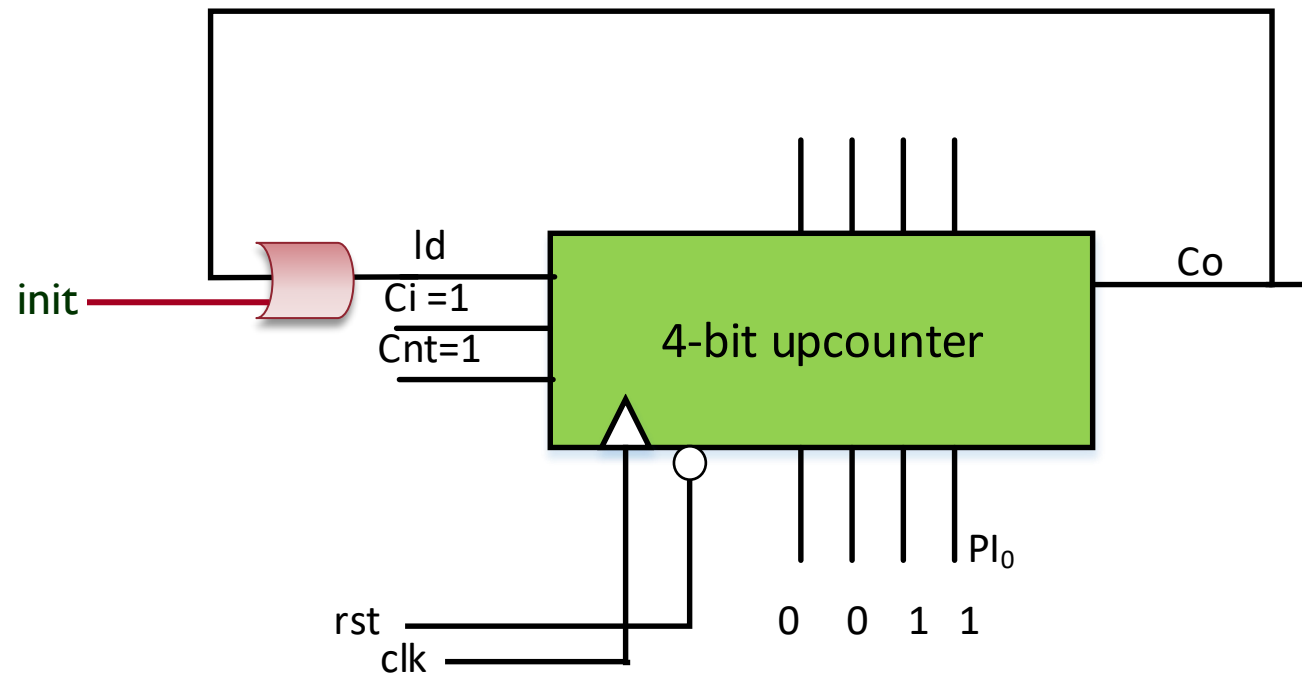
- 16 with outputs available at CO and PO[3]
- 8 with the PO[2] output
- 4 with the PO[1] output
- 2 with the PO[0] output



Sequential components

Frequency divider by 13 (Functionality)

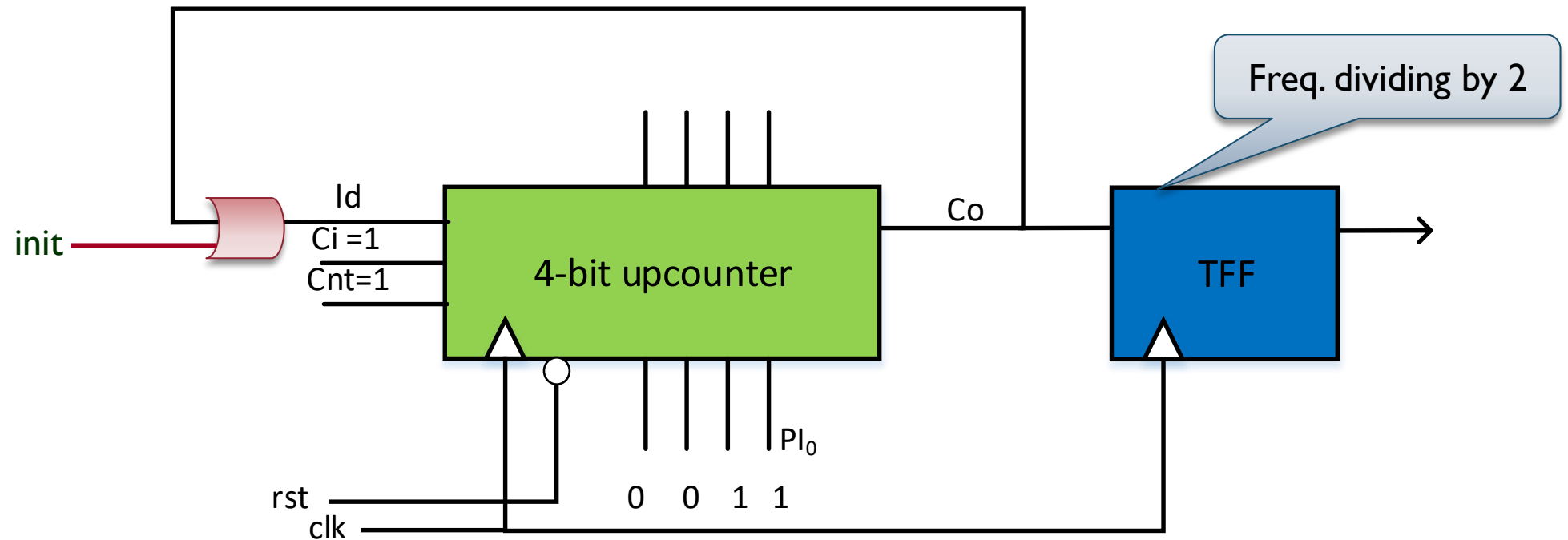
- Frequency division by 13 equals to multiplying the time by 13.
- That equals to counting for 13 clock cycles. So the parallel input must be set to $16-13=3$.



Sequential components

Frequency divider by 13 (Functionality)

- When Co becomes one after reaching 15, then the counter must be reinitialized by issuing the ld signal.
- A T- flip flop is used to generate a 50% duty cycle.



Sequential components

3- Sequential Components

3-0 Recall

3-1 Registers

3-2 Shift Register

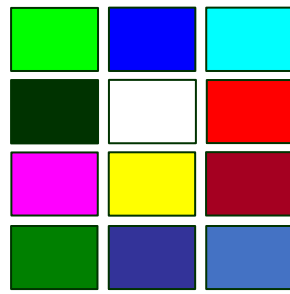
3-3 Counters

3-3-1 Up-counter

3-3-2 Frequency divider

3-3-3 Counter cascading

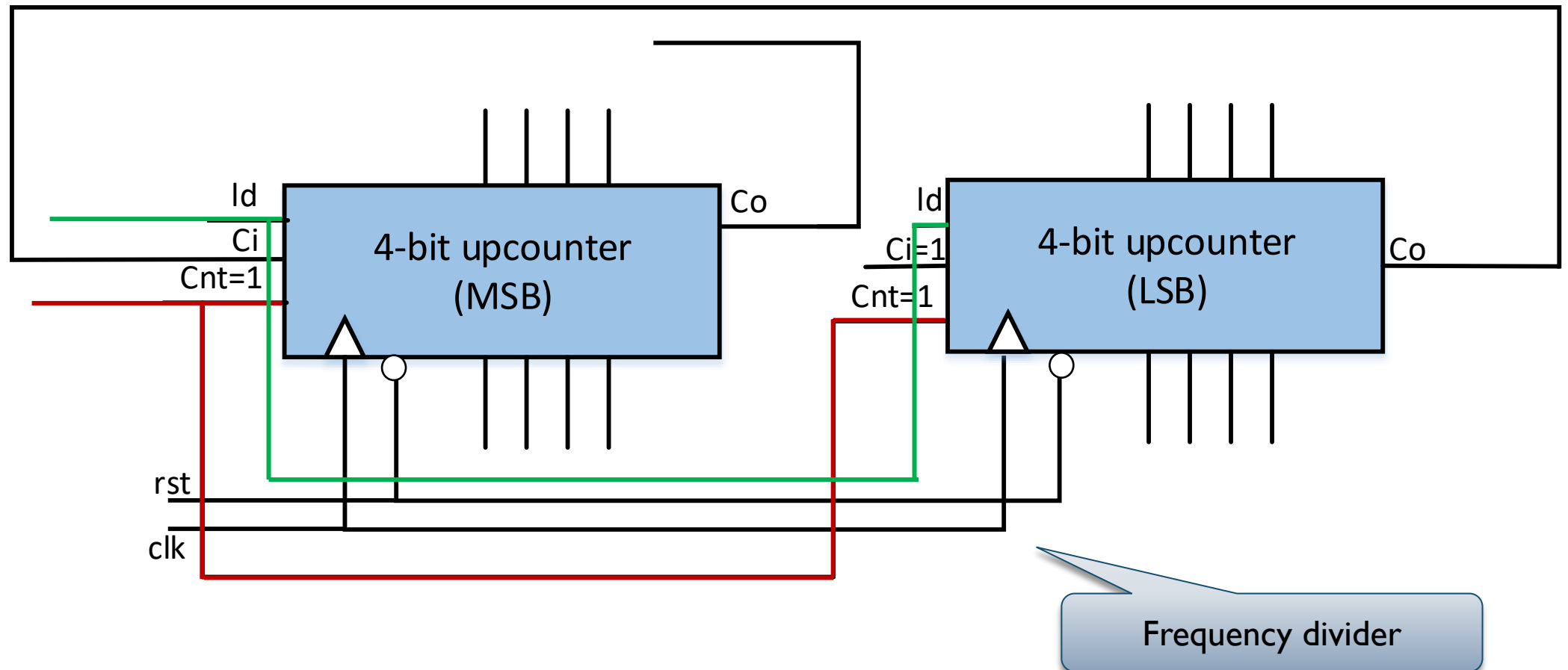
3-4 Register file



Sequential components

Counter cascading

8-bit Counter using 4-bit counters (Functionality)

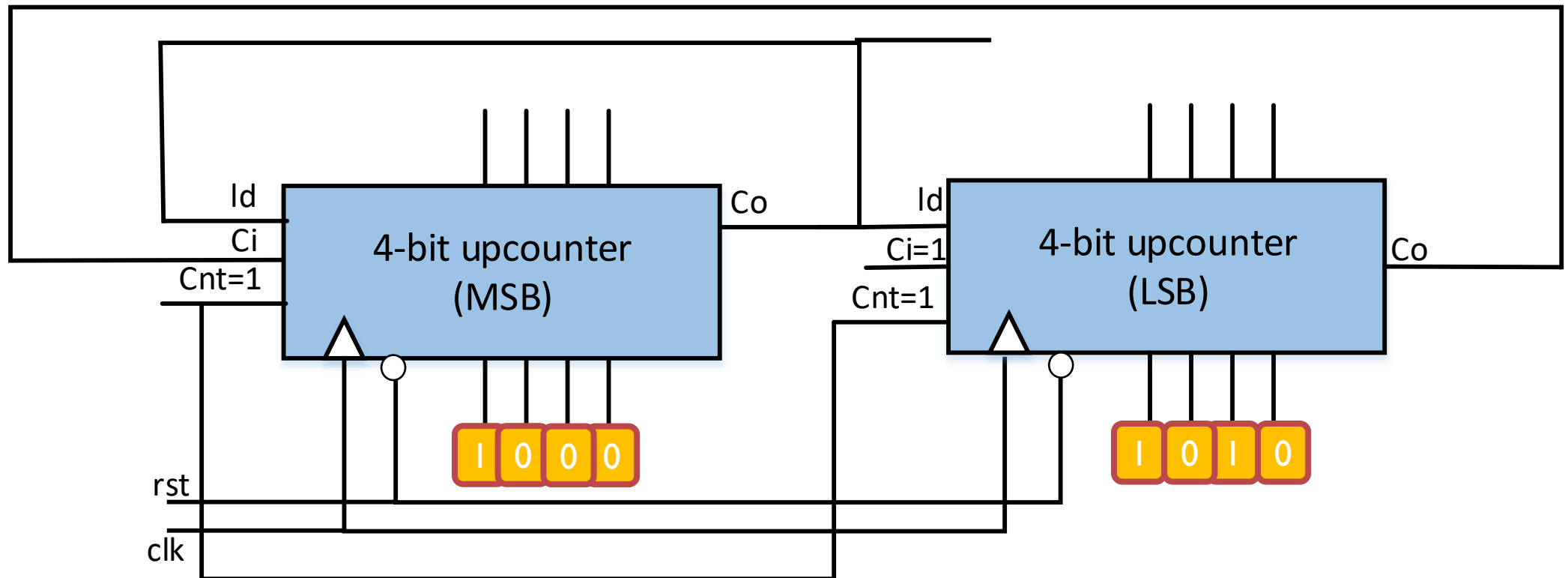


Sequential components

Counter cascading

8-bit frequency divider using 4-bit counters (Functionality)

For division by 118, $PI = 256 - 118 = 138$



Sequential components

Counter cascading

Alternating between dividing 118 and 238

For division by 118
 $PI = 256 - 118 = 138$

For division by 238
 $PI = 256 - 238 = 18$

$$(PI)_{D-118} = 10001010$$
$$(PI)_{D-238} = 00010010$$

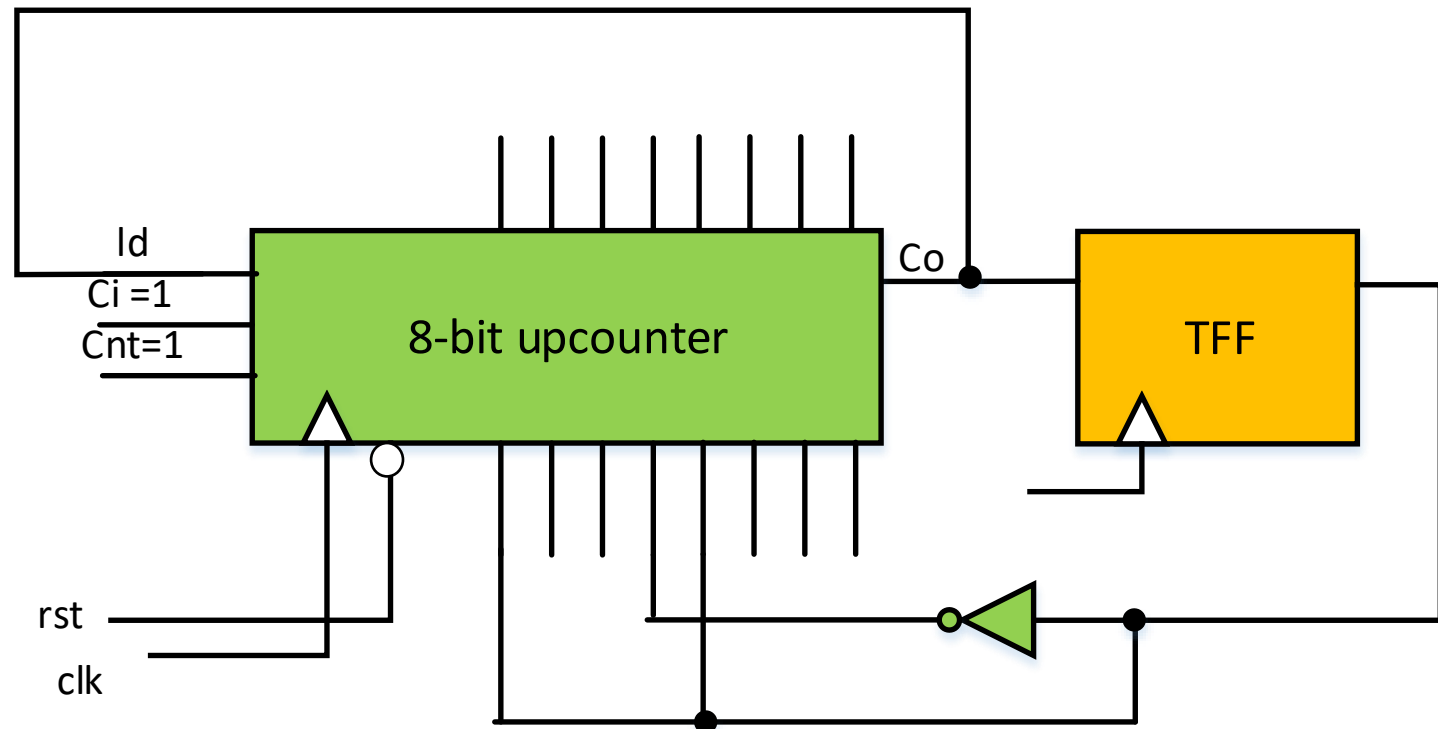
Sequential components

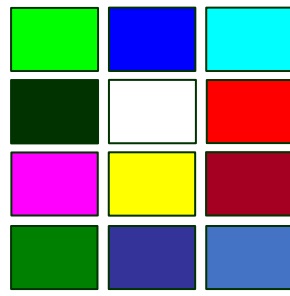
Counter cascading

Alternating between dividing 118 and 238

$$(PI)_{D-118} = 10001010$$

$$(PI)_{D-238} = 00010010$$





Sequential components

3- Sequential Components

3-0 Recall

3-1 Registers

3-2 Shift Register

3-3 Counters

3-3-1 Up-counter

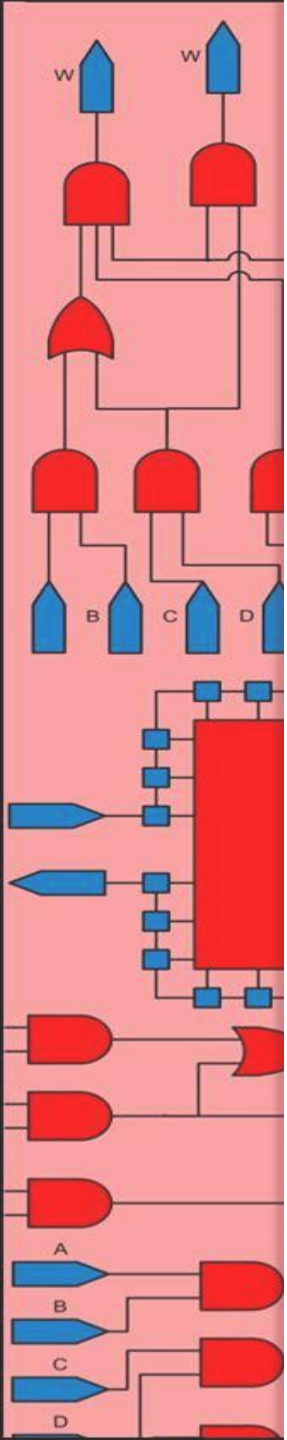
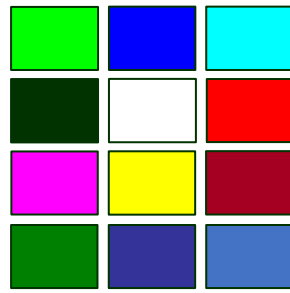
3-3-2 Frequency divider

3-3-3 Counter cascading

3-4 Register file

Sequential components

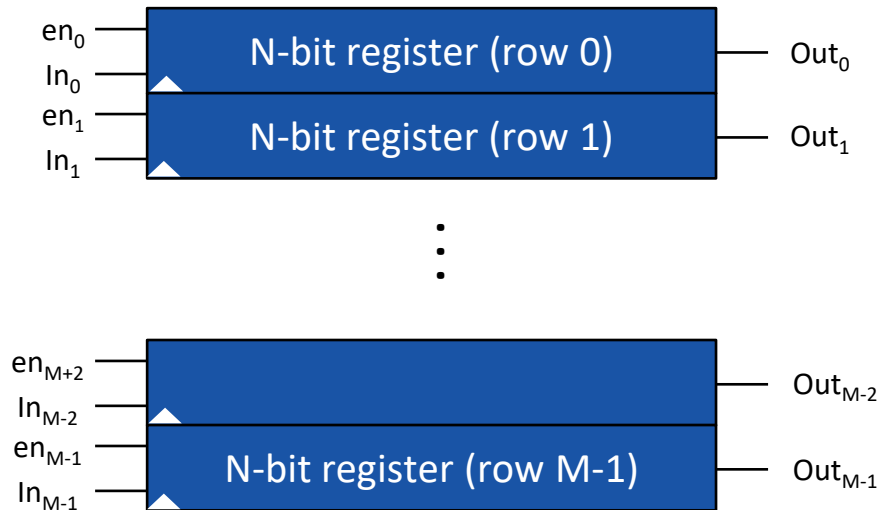
Register File



Sequential components

Register File

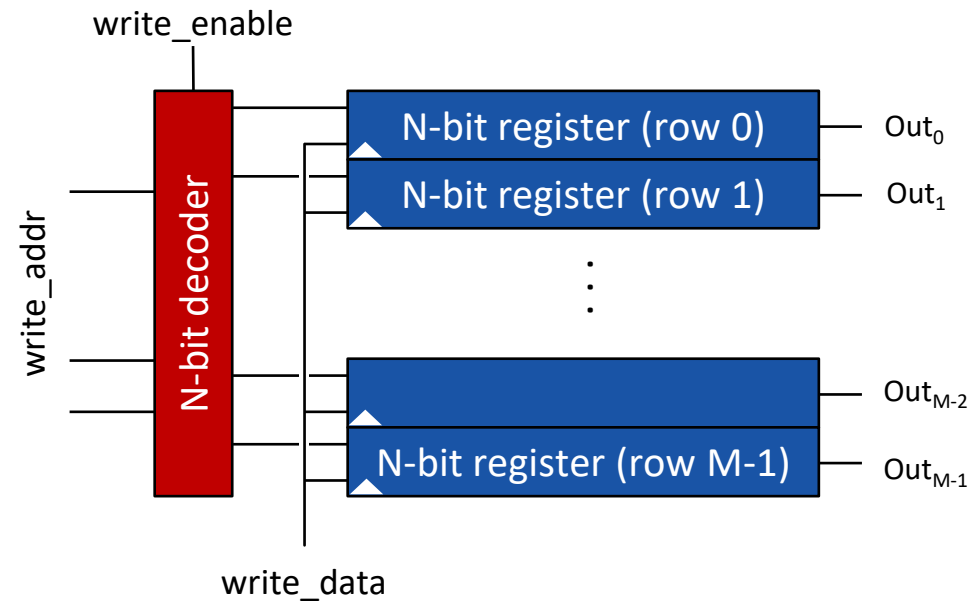
- An array of N-bit registers (M rows)



Sequential components

Register File

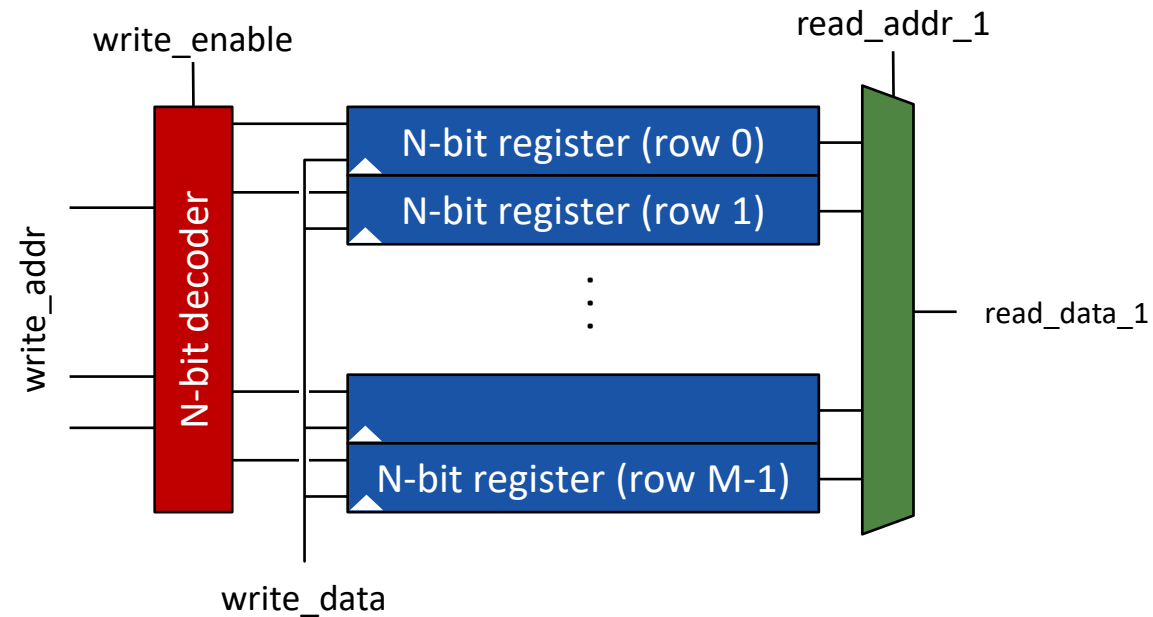
- An array of N-bit registers (M-rows)
- Writing to the registers requires a decoder to assign the correct enable signals



Sequential components

Register File

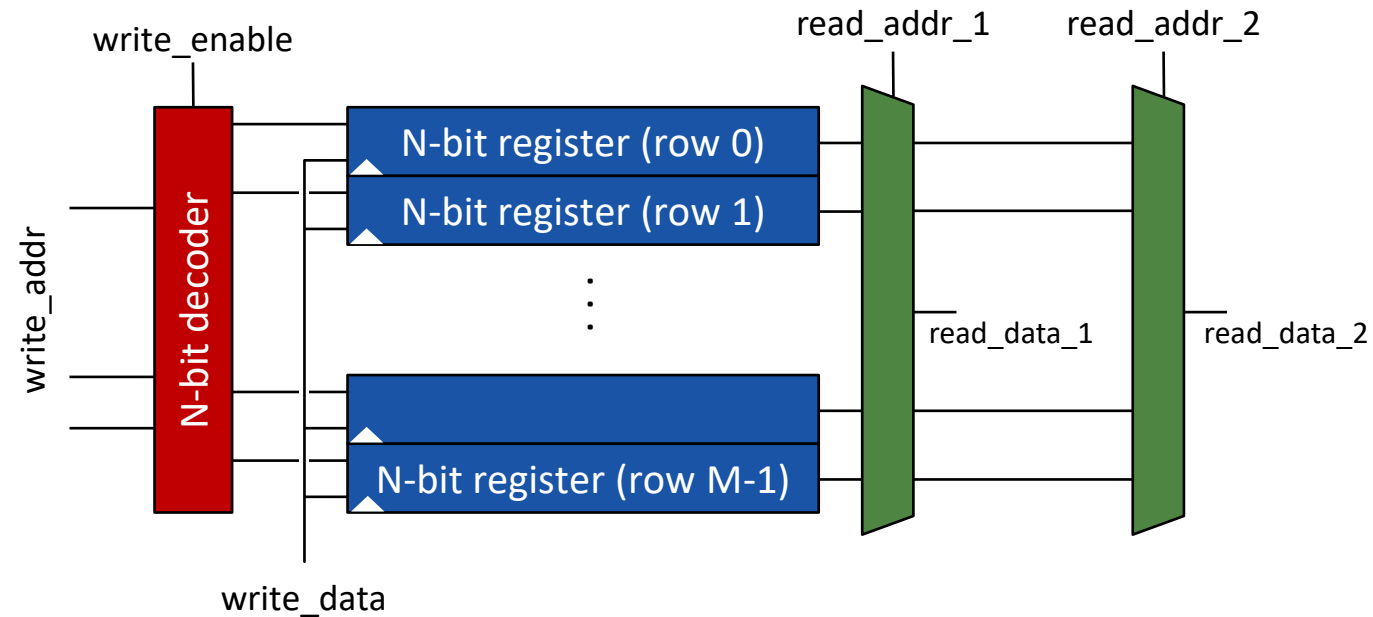
- An array of N-bit registers (M-rows)
- Writing to the registers requires a decoder to assign the correct enable signals
- Reading requires a multiplexer to select the correct output



Sequential components

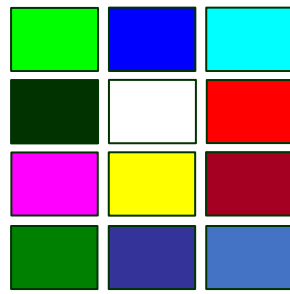
Multiport Register File

- An array of N -bit registers (M -rows)
- Writing to the registers requires a decoder to assign the correct enable signals
- Reading requires multiple multiplexers to select the correct outputs



Sequential components

Multiport Register File



```
module register_file #(
    parameter DATA_WIDTH = 32,
    parameter NUM_REGS = 16
) (
    input logic                clk,
    input logic                rst_n,
    input logic [4:0]          read_addr_1,
    input logic [4:0]          read_addr_2,
    input logic [4:0]          write_addr,
    input logic [DATA_WIDTH-1:0] write_data,
    input logic                write_enable,
    output logic [DATA_WIDTH-1:0] read_data_1,
    output logic [DATA_WIDTH-1:0] read_data_2
);
```

Read logic (Asynchronous)

Write logic (Synchronous)

Internal register array

```
logic [DATA_WIDTH-1:0] regs [NUM_REGS];
```

```
assign read_data_1 = regs[read_addr_1];
assign read_data_2 = regs[read_addr_2];
```

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        for (int i = 0; i < NUM_REGS; i++) begin
            regs[i] <= '0;
        end
    end else if (write_enable) begin
        regs[write_addr] <= write_data;
    end
end
```

```
endmodule
```

Conclusion

We have covered these topics:

- Sequential components
- Registers
- Shift Register
- Counters
- Register file

